

Client Hints is an extension to the HTTP protocol that allows servers to ask the client (usually a web browser) for information about its configuration. This helps the server tailor its responses to the client; for example, a server can choose to send a smaller image if a client advertises that they have a very small screen. The client can choose to respond to this request by advertising the requested information about itself by sending the data using a specific part of the HTTP protocol called HTTP header fields or by exposing the same information to the JavaScript code being executed on a web page.

Proposed by Google engineers in 2013, Client Hints was designed as a privacy-focused alternative to user-agent headers. This was done as part of an initiative by Google called Privacy Sandbox. User-agent headers are code snippets sent by a client to identify itself to a server. While initially intended for statistical purposes, these headers had increasingly become a tool for tracking users across websites. Client Hints aimed to address this issue by providing a more controlled way to share the same information. Despite the focus on privacy, the initial design of Client Hints faced criticism from other browsers. One of the primary concerns that was brought up was that the protocol could enable new forms of tracking by third-party domains. Third-party domains are web servers not owned by the website that load resources like images and script files. Despite these concerns, Chrome implemented support for Client Hints in August 2020. By 2024, over 75% of web users had browsers that supported Client Hints.

Privacy researchers have since raised concerns that Client Hints is primarily being used by JavaScript code which tracks users. In 2023, a study from KU Leuven and Radboud University found that when examining the top 100,000 websites on the internet, most accesses of Client Hints came from JavaScript code used for tracking and advertising purposes.

Background

In 1992, an extension to the HTTP protocol was introduced adding a User-Agent HTTP Header

which was sent from the client to the server and contained a simple string identifying the name of the client and its version. The header was meant purely for statistical purposes and for tracking down clients that violated the protocol. Since then, User-Agent headers have become increasingly more complex, and has started containing significant uniquely identifiable information about the user. Often, this information is used to perform browser fingerprinting, allowing sites to track users across sites passively without having to load any JavaScript for the user.

History

The original draft for the Client Hint specification was proposed in 2013 by engineers at Google. The specifications became an Internet Engineering Task Force (IETF) draft in November 2015. In 2021, the specification was upgraded to the status of an experimental request for comment (RFC). This designation indicated that the IETF had accepted the Client Hints specification as an internet standard, but it either still had unresolved questions or had not yet gained widespread adoption in the internet. Around the same time, the specifications for how web browser would be handling HTTP Client Hints on the web was published as a draft in a W3C Community Group Report.

In 2020, Google announced their intention to deprecate user-agent (UA) declaration by the browser. This deprecation was part of a broader initiative by Google to make changes to the web that allow websites to access user information without compromising privacy called Privacy Sandbox. They cited Client Hints as a privacy-preserving alternative to user-agent headers since they allowed for a more controlled way of sharing the same information. The initial Client Hints proposal, however, was met with pushback from other browsers due to privacy concerns. In 2019, Brave raised concerns about the initial proposal, citing ways in which it could be used to track users on the internet. Mozilla, the company that makes Firefox, initially classified the proposal as harmful, and Apple, the company that makes Safari also took

a negative stance against the proposal. Despite these concerns, Chrome implemented support for HTTP Client Hints in August 2020. While the deprecation of the UA strings was delayed due to the COVID-19 pandemic, this process was completed in February 2023.

Since their initial opposition, Mozilla has updated their stance to neutral and Brave has synchronized its implementation of Client Hints with that of Chrome. As of 2024, over 75% of all web users use browsers that support Client Hints.

Mechanism

The Client Hints protocol defines two entities: a user agent (UA) (typically a browser) and a server. These two entities communicate with each other to negotiate what kind of content should be served to the user. The process involves the server sending the UA a response with an Accept-CH HTTP Header, containing a list of Client Hint HTTP headers that it requires. Subsequently, the UA is expected to return the requested client hints with each subsequent response, provided it supports those hints. These headers are then used by the server to make decisions on what kind of content to serve the UA. If the UA does not understand or support a particular client hint then the UA is instructed to ignore the particular client hint. In cases where a specific Client Hint cannot be cached, the server must specify the applicable client hints headers in a separate Vary header sent to the UA. This ensures that caching mechanisms understand that responses can vary based on different client hint values. For client hints that specifically identify a browser, additional random browser identifiers are included as grease in order to prevent users of the protocol from relying on browser specific idiosyncratic behaviours.

For UAs that allow JavaScript, an additional option is available through the navigator.userAgentData JavaScript API. This API enables JavaScript to retrieve the same information as provided by the Client Hints headers. The API separates the data it provides into two types: low-entropy data and high-entropy data. Low-entropy data corresponds to

information that is likely to be similar across a large group of users, such as the platform on which the browser is running and the brand of the browser. In contrast, high-entropy data may vary significantly between users, including details like the exact version number of the browser and the model of the user's device. Low entropy data is included in the API as object parameters whereas high entropy data which can uniquely identify the user needs to be explicitly fetched by the client by calling the `getHighEntropyValues()` function in the API which allows the browser to ask for user permission or to perform additional checks.

Example

To initiate a content negotiation, a HTTP server appends the `Accept-CH` header to the response of a HTTP request: If the user-agent supports the viewport width client hint, the user-agent will append the `Viewport-Width` header in every subsequent request. The server can then use the information in the `Viewport-Width` header to make a decision about the kind of content to serve the client. For example, if the server has a particular image that is extremely large, the server can be configured to return a smaller image if the image does not fit the viewport.

Privacy concerns

When the Client Hints proposal was originally published, it was met with significant privacy concerns. Browser vendors like Brave and Mozilla pointed out that a particular provision in the initial draft of the proposal allowed websites to instruct the browser to provide Client Hint data to third-party domains. Third-party domains are domains that do not execute any JavaScript code, but rather load resources like images and script files. The provision in the initial draft would allow these third-party domains like content delivery networks (CDNs) and cloud service providers. CDNs distribute website content across a network of geographically dispersed group of servers to improve the speed and reliability of the website. Cloud

providers like Cloudflare and Google Cloud offer services like data storage, computing power, and infrastructure for websites and applications. These entities could track users across the web by instructing the browser to send Client Hint information to their servers alongside the original website. Concerns were also raised that the Client-Hint proposal was too permissive and explicitly allowed for new privacy compromising information that could not be obtained by simply reading HTTP Headers to be leaked to servers. Additionally, extensions that aim to preserve a user's privacy like the NoScript extension also opposed the proposal on the grounds that it would make it significantly harder to prevent sites from exfiltrating privacy-compromising information about users.

Since the adoption of Client Hints by major browsers like Google Chrome and Microsoft Edge, privacy researchers have raised concerns over their real-world use for tracking. A 2023 study by researchers from KU Leuven and Radboud University found that out of the top 100,000 websites, 60% of JavaScript files loaded by web pages accessed the Client Hints JavaScript APIs, with most being tracking and advertising scripts, many of which came from Google. Over 90% of these script files exfiltrated the obtained data to tracking domains. A subsequent study in May 2024 by researchers from the Hochschule Bonn-Rhein-Sieg University of Applied Sciences noted that while overall adoption of Client Hints amongst websites on the internet was low, a significant number of third-party domains known for tracking accessed HTTP Client Hints data.

See also

Browser sniffing

References

Citations

Sources

External links

